❷ Runs before any other action filters for every action in the controller

❸ Runs after all other action filters for every action in the controller

If you override these methods on your controller, they'll run in the action filter stage of the filter pipeline for every action on the controller. The `OnActionExecuting` method runs before any other action filters, regardless of ordering or scope, and the `OnActionExecuted` method runs after all other action filters.

TIP The controller implementation can be useful in some cases, but you can't control the ordering related to other filters. Personally, I generally prefer to break logic into explicit, declarative filter attributes, but it depends on the situation, and as always, the choice is yours.

With the resource and action filters complete, your controller is looking much tidier, but there's one aspect in particular that would be nice to remove: the exception handling. In the next section we'll look at how to create a custom exception filter for your controller and why you might want to do this instead of using exception handling middleware.

22.1.4 Exception filters: Custom exception handling for your action methods

In chapter 4 I went into some depth about types of error-handling middleware you can add to your apps. These let you catch exceptions thrown from any later middleware and handle them appropriately. If you're using exception handling middleware, you may be wondering why we need exception filters at all.

The answer to this is pretty much the same as I outlined in chapter 21: filters are great for cross-cutting concerns, when you need behavior that's specific